

Audspect BAS Platform

Product Guide — v1.6.0



Classification: Confidential — Customer Distribution
Document Version: 1.0
Platform Version: 1.6.0
Prepared by: Audspect
Contact: support@audspect.com

Table of Contents

- 1. [Executive Summary](#)
- 2. [Product Overview](#)
- 3. [Architecture Overview](#)
- 4. [System Requirements](#)
- 5. [Features & Capabilities](#)
- 6. [MITRE ATT&CK Coverage](#)
- 7. [Scenario Library](#)
- 8. [Scoring & Risk Methodology](#)
- 9. [User Roles & Access Control](#)
- 10. [Installation & Deployment](#)
- 11. [Agent Deployment](#)
- 12. [Dashboard & Operations Guide](#)
- 13. [API Reference](#)
- 14. [Configuration Reference](#)
- 15. [Security & Compliance Posture](#)
- 16. [Licensing](#)
- 17. [Troubleshooting](#)
- 18. [Glossary](#)

1. Executive Summary

Audspect BAS (Breach and Attack Simulation) is an enterprise-grade, on-premises security validation platform designed for BFSI (Banking, Financial Services, and Insurance) organizations and other regulated industries. The platform continuously validates the effectiveness of security controls by simulating real-world adversary techniques across production and staging environments — without disrupting operations.

Unlike vulnerability scanners that identify weaknesses in software, Audspect BAS tests whether deployed security controls (EDR, AV, SIEM, DLP, firewall policies, hardening baselines) actually stop or detect known

attack patterns. Results are mapped to the MITRE ATT&CK framework, scored across multiple risk dimensions, and surfaced in a real-time dashboard built for security operations teams.

Key differentiators:

- Fully air-gapped, on-premises deployment — no data leaves the customer environment
- Native BFSI scenario coverage (UPI fraud kill-chain, RBI/CSCRF drill scenarios, APT36 spear-phishing)
- Multi-framework orchestration: custom posture checks, Atomic Red Team, and Caldera from a single interface
- Lightweight cross-platform agents (Windows, Linux, macOS) with a 50 MB footprint
- Real-time WebSocket-driven dashboard with per-tactic kill-chain scoring

2. Product Overview

2.1 What Audspect BAS Does

Audspect BAS deploys a lightweight agent on endpoint machines (servers, workstations, cloud VMs) and a central orchestrator server within the customer's network. Security analysts use the web dashboard to select pre-built attack scenarios or create custom ones, then dispatch them to one or more agents with a single click.

Each agent executes the scenario's attack steps using native OS tooling — PowerShell, cmd.exe, Bash, WMI — and reports the raw output back to the orchestrator. The orchestrator interprets results against the MITRE ATT&CK framework, computes a multi-dimensional risk score, and streams the results live to the dashboard.

2.2 What It Is Not

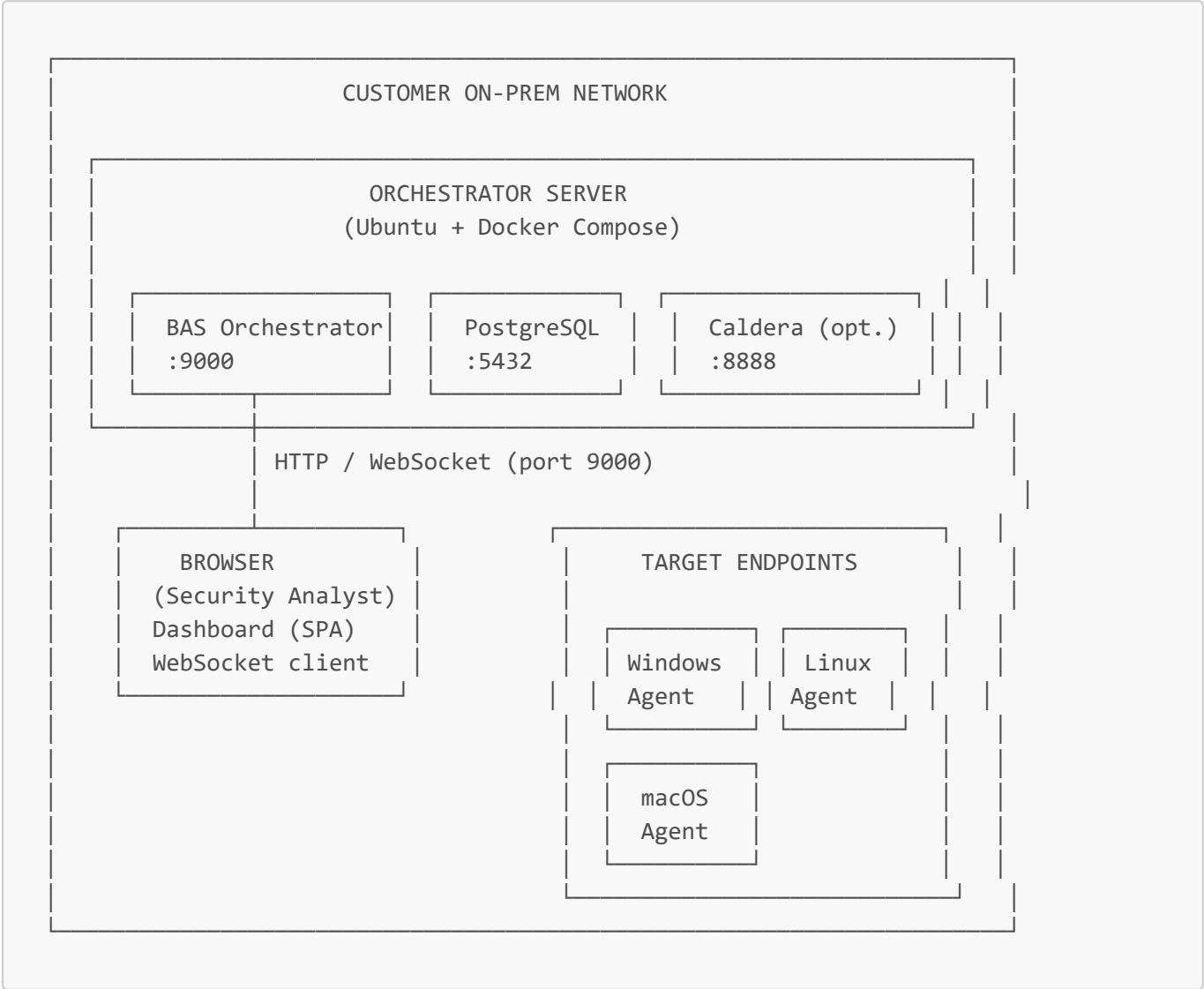
- **Not a vulnerability scanner.** Audspect BAS does not scan for CVEs or patch-level weaknesses; it tests whether your controls stop known attack behaviors.
- **Not a SIEM.** It does not ingest logs; it generates ground-truth attack events that your SIEM should be detecting.
- **Not an agent-based EDR replacement.** The agent is exclusively an execution and reporting vehicle — it carries no threat intelligence and performs no ongoing monitoring.

2.3 Platform Components

Component	Description	Deployment
Orchestrator	Go-based backend; scenario engine, scoring, API, WebSocket hub, web dashboard	Docker container on Ubuntu server
PostgreSQL	Persistent store for agents, runs, reports, users	Docker container (bundled)
Web Dashboard	Single-page application served from the orchestrator	Browser (no installation)
BAS Agent	Cross-platform binary; executes scenarios, sends results	Installed on each target endpoint

Component	Description	Deployment
Caldera <i>(optional)</i>	Adversary emulation framework; used as ability source	Docker container (optional)

3. Architecture Overview



3.1 Communication Flow

- Agent → Orchestrator** (outbound from endpoint): WebSocket connection on port 9000. Agents initiate the connection; no inbound ports required on endpoints.
- Heartbeat:** Every 30 seconds, the agent sends its status (idle/scanning), hostname, IP, OS version, and binary hash.
- Scenario Dispatch:** The analyst selects a scenario and target agent. The orchestrator sends a `command_scenario` message over the established WebSocket.
- Result Submission:** After execution, the agent posts raw results via HTTPS to `/api/scenarios/result`. Each result is signed with an HMAC-SHA256 MAC to prevent tampering.
- Live Streaming:** The orchestrator interprets results and broadcasts `scenario_result` events over WebSocket to connected browser clients in real time.
- Browser → Orchestrator:** Standard HTTPS (REST API) and WebSocket for live updates.

3.2 Data Residency

All data — agents, results, reports, user accounts — is stored exclusively in the on-premises PostgreSQL instance. No telemetry, results, or agent data is transmitted to Audspect or any external service.

4. System Requirements

4.1 Orchestrator Server

Requirement	Minimum	Recommended
OS	Ubuntu 20.04 LTS	Ubuntu 22.04 / 24.04 LTS
CPU	2 vCPU	4 vCPU
RAM	4 GB	8 GB
Disk	40 GB	100 GB (for large report history)
Docker	24.0+	25.0+
Docker Compose	v2.20+	v2.24+
Network	Reachable from all endpoints	Static LAN IP recommended
Port	9000 (HTTP/WS, inbound from LAN)	Reverse-proxy for TLS

4.2 BAS Agent — Windows

Requirement	Value
OS	Windows 7 SP1 / Server 2008 R2 or later
Architecture	x86_64 (amd64)
PowerShell	3.0 or later
Privileges	Local Administrator (for service installation and full scenario coverage)
RAM	~50 MB resident
Disk	~20 MB binary
Network	Outbound TCP 9000 to orchestrator

4.3 BAS Agent — Linux

Requirement	Value
OS	Ubuntu 18.04+, CentOS 7+, RHEL 8+, Debian 10+
Architecture	x86_64 (amd64), ARM64
Shell	bash or sh
Privileges	root (for service installation)

Requirement	Value
Package	.deb (Ubuntu/Debian), .rpm (RHEL/CentOS), raw binary
RAM	~30 MB resident
Network	Outbound TCP 9000 to orchestrator

4.4 BAS Agent — macOS

Requirement	Value
OS	macOS 10.12 (Sierra) or later
Architecture	x86_64 (Intel), ARM64 (Apple Silicon)
Shell	bash
Privileges	Administrator account
Network	Outbound TCP 9000 to orchestrator

4.5 Browser (Dashboard)

Requirement	Value
Browser	Chrome 90+, Firefox 88+, Edge 90+, Safari 14+
JavaScript	Required (enabled)
WebSocket	Required (not blocked by proxy)
Resolution	1280×720 minimum; 1920×1080 recommended

4.6 Optional: Caldera Integration

Requirement	Value
Caldera Version	5.0+
Access	REST API reachable from orchestrator container
Auth	API key configured in orchestrator environment

5. Features & Capabilities

5.1 Scenario Orchestration

The scenario engine is the core of the platform. It translates high-level scenario definitions (YAML files) into executable steps, resolves framework-specific commands server-side, and dispatches pre-built payloads to agents. The agent is intentionally kept "dumb" — it receives fully resolved commands and executors; no framework SDKs or ART modules are required on the endpoint.

Supported execution frameworks:

Framework	Description	Requires on Endpoint
Custom	PowerShell/Bash posture checks with PASS/FAIL/SKIP output	Nothing
ART	Atomic Red Team technique execution	Nothing (server bundles YAML)
Caldera	Caldera adversary emulation abilities	Nothing (server fetches via API)

Framework resolution priority (checked in order for each scenario):

1. **local_check: true** — Agent runs built-in hardening checks
2. **caldera_adversary_id** — Fetch all abilities in a named Caldera adversary profile
3. **caldera_abilities** — Execute specific Caldera ability IDs
4. **caldera_all_windows** — Execute all Caldera Windows abilities
5. **art_techniques** — Execute specific ART technique IDs
6. **art_all_windows** — Execute all ART Windows techniques
7. **steps** — Static YAML steps (always available, no external dependencies)

5.2 Multi-Platform Agent

The BAS agent runs on Windows, Linux, and macOS from a single codebase. It supports the following executor types per platform:

Windows executors: `powershell`, `cmd`, `wmi`, `mshta`, `rundll32`, `cscript`, `regsvr32`, `schtasks`

Linux/macOS executors: `bash`, `sh`

Agents register as a system service (`--install`) and survive reboots. They reconnect automatically after network interruptions using exponential backoff.

5.3 Real-Time Dashboard

The web dashboard is a fully client-side single-page application served from the orchestrator. It requires no separate frontend server. Key views:

- **Dashboard:** Live agent status overview, run history, recent results
- **Scenarios:** Browse all available scenarios with MITRE tactic tags
- **Live Runs:** Real-time stream of executing scenario steps and results
- **Agents:** All registered endpoints with OS, IP, status, trust state
- **Reports:** Per-agent full simulation reports with scoring breakdown
- **Users (Admin):** Create, edit, and deactivate user accounts
- **Settings (Admin):** Platform configuration

5.4 Scoring Engine

Every scenario run produces a multi-dimensional Score. See [Section 8](#) for the full methodology.

5.5 Native Linux Packaging

Linux agents are distributed as native system packages:

Format	Architectures	Package Manager
.deb	amd64, arm64	apt / dpkg
.rpm	x86_64	yum / dnf / rpm
Raw binary	amd64, arm64	Manual

All packages include a systemd unit (**bas-agent.service**) and an interactive post-install prompt that captures the orchestrator URL and agent secret securely (password is masked during input).

5.6 Agent Secret & Connection Configuration

Administrators can view the agent secret and server URL directly from the dashboard (Admin → Connection Config card). This eliminates the need for SSH access to distribute credentials. The secret is masked by default with Show/Copy controls.

5.7 Binary Integrity Verification

On every heartbeat, the orchestrator optionally verifies the connecting agent's binary hash against a **BINARIES.sha256** manifest shipped with the platform. Untrusted binaries are flagged in the dashboard and in server logs, enabling detection of tampered or custom agent builds.

5.8 HMAC-Verified Result Submission

All scenario results and reports submitted by agents are signed with HMAC-SHA256 using the shared **AGENT_SECRET**. The orchestrator rejects any result where the MAC does not match, preventing injection of false results by network-based attackers.

5.9 Agent Staleness Detection

A background monitor checks every 30 seconds for agents that have not sent a heartbeat in over 90 seconds. These agents are automatically marked **Offline** and any in-progress scenario runs are marked **Partial** (rather than hanging indefinitely). Connected dashboards receive the update in real time.

5.10 User & Session Management

- JWT-based authentication (HS256, 24-hour TTL)
- HttpOnly, SameSite=Strict cookies (CSRF-resistant)
- Bcrypt password storage (min 8 characters)
- Forced password change on first login
- Admin-initiated password reset for any user
- User deactivation without deletion (preserves audit trail)

5.11 Air-Gapped Deployment

The full delivery package includes Docker image tarballs (**bas-orchestrator-<version>.tar**, **postgres-16-alpine.tar**) and optionally **caldera-latest.tar**. No internet access is required on the server after deployment. The ART atomics YAML library is bundled inside the orchestrator image at build time.

6. MITRE ATT&CK Coverage

The platform maps every simulation step to a MITRE ATT&CK technique. The following tactics are covered:

Tactic	Weight	Example Techniques
Credential Access	Critical	T1003 (LSASS, SAM, NTDS), T1552, T1555
Lateral Movement	Critical	T1021 (RDP, SMB), T1550 (Pass-the-Hash)
Privilege Escalation	Critical	T1134 (Token Manipulation), T1055 (Process Injection), T1548
Persistence	High	T1547 (Registry Run Keys), T1543 (Services), T1053 (Scheduled Tasks)
Defense Evasion	High	T1562 (Disable/Modify Tools), T1070 (Indicator Removal), T1027
Execution	High	T1059 (PowerShell, Cmd), T1047 (WMI), T1204
Command & Control	High	T1071, T1095, T1572
Impact	High	T1486 (Ransomware), T1490, T1498
Exfiltration	High	T1041, T1048, T1567
Collection	High	T1560, T1113 (Screenshot), T1114 (Email)
Initial Access	Medium	T1566 (Phishing), T1190, T1133
Discovery	Low	T1087, T1082, T1018, T1083
Reconnaissance	Low	T1595, T1592

Total coverage: 60+ base techniques, 200+ sub-techniques, 13 tactics.

The platform includes a dedicated **TechniqueNameMap** and **TacticMap** for automatic resolution of technique names and tactic assignments from technique IDs, ensuring consistent labeling even when scenarios specify only a technique ID.

7. Scenario Library

The platform ships with 12 pre-built scenarios. Additional scenarios can be added by dropping YAML files into the scenarios directory (no restart required).

7.1 Built-in Scenarios

Scenario ID	Name	Framework	Focus
full-scan	Full Posture Scan	Custom (local check)	Comprehensive Windows hardening baseline
art-full-windows	ART Full Windows Sweep	ART (all)	Complete ART Windows technique library

Scenario ID	Name	Framework	Focus
art-selective	ART Targeted Techniques	ART (selective)	Specific high-priority techniques
caldera-full-windows	Caldera Full Windows	Caldera (all)	All Caldera Windows abilities
caldera-selective	Caldera Targeted Abilities	Caldera (selective)	Specific ability IDs
caldera-lateral-movement	Lateral Movement Drill	Caldera (adversary)	Named Caldera adversary profile
ad-credential-access	AD Credential Access	Custom/ART	Active Directory attack paths
apt36-spearphish	APT36 Spear-Phishing	Custom	South Asia APT36 TTPs
cis-ubuntu-l1	CIS Ubuntu L1 Benchmark	Custom	Linux hardening benchmark
cscrf-mii-drill	CSCRF/MII Compliance Drill	Custom	SEBI/RBI financial sector controls
lolbin-execution	LOLBIN Execution	Custom/ART	Living-off-the-land binaries
ransomware-drill	Ransomware Response Drill	Custom	Ransomware defense controls
upi-fraud-killchain	UPI Fraud Kill Chain	Custom	UPI payment fraud detection

7.2 Full Posture Scan (Detail)

The **full-scan** scenario is the recommended starting point for all new deployments. It runs 50+ PowerShell checks across the following categories:

Credential Protection

- RunAsPPL (LSASS protection)
- WDigest disabled
- Credential Guard enabled
- LSA Additional Protection

Defense Controls

- Windows Defender real-time protection
- Windows Firewall (Domain, Private, Public profiles)
- UAC enabled and at appropriate level
- AMSI (Antimalware Scan Interface)
- SmartScreen enforcement

Execution Controls

- PowerShell Constrained Language Mode
- PowerShell Script Block Logging
- Module Logging
- Transcription Logging

Persistence Controls

- Suspicious auto-start registry entries
- Scheduled task anomalies

Lateral Movement Controls

- NTLM relay restrictions (SMB signing, LDAP signing)
- RDP NLA enforcement

Privilege Escalation Controls

- UAC bypass protections
- Token manipulation restrictions

7.3 Custom Scenario YAML Format

```
id: my-custom-scenario
name: My Custom Scenario
description: Tests specific control set
author: Security Team
tags: [credential-access, windows]
mitre_phases: [credential-access, execution]

steps:
  - name: Check LSASS RunAsPPL
    technique_id: T1003.001
    framework: custom
    executor: powershell
    timeout_sec: 30
    command: |
      $val = (Get-ItemProperty "HKLM:\SYSTEM\CurrentControlSet\Control\Lsa" -
ErrorAction SilentlyContinue).RunAsPPL
      if ($val -eq 1) { "PASS: RunAsPPL is enabled" } else { "FAIL: RunAsPPL is
not enabled" }
```

Custom check output convention:

- **PASS:** <reason> — Control is working
- **FAIL:** <reason> — Control is missing or ineffective
- **SKIP:** <reason> — Check not applicable to this endpoint

8. Scoring & Risk Methodology

Every completed scenario run produces a **Score** object with five dimensions. The methodology is designed for BFSI security operations, where a single high-value failure (e.g., credential dump succeeds) carries more weight than multiple low-severity failures.

8.1 Prevention Score (0–100, higher = safer)

Measures the weighted pass rate across all executed checks.

$$\text{PreventionScore} = \frac{\sum(\text{weight}(\text{severity}) \times \text{passed})}{\sum(\text{weight}(\text{severity}) \times \text{executed})} \times 100$$

Severity weights:

Severity	Weight
Critical	4
High	3
Medium	2
Low	1

A score of 100 means every check passed, weighted by severity. A score of 0 means every check failed.

8.2 Exposure Score (0–100, higher = worse)

Measures the tactic-weighted fail rate, amplified by how far an attacker could advance through the kill chain.

$$\begin{aligned}\text{RawExposure} &= \frac{\sum(\text{tacticWeight} \times \text{failedInTactic})}{\sum(\text{tacticWeight} \times \text{executedInTactic})} \times 100 \\ \text{ExposureScore} &= \text{RawExposure} \times \text{KillChainAmplifier}\end{aligned}$$

Tactic exposure weights:

Weight	Tactics
4	credential-access, lateral-movement, privilege-escalation
3	persistence, defense-evasion, execution, command-and-control, impact, exfiltration, collection
2	initial-access
1	discovery, reconnaissance

8.3 Kill Chain Amplifier (1.0–2.5×)

When an attacker can advance through consecutive kill-chain phases without being stopped, each additional phase multiplies the risk. The amplifier is applied to the Exposure Score.

Consecutive Failing Phases	Amplifier
0–1	1.0×
2	1.3×
3–4	1.8×
5+	2.5×

Kill-chain phase order used for amplifier calculation: Initial Access → Execution → Persistence → Privilege Escalation → Defense Evasion → Credential Access → Lateral Movement → Collection → Exfiltration

8.4 Coverage Score (0–100, higher = safer)

The percentage of tested ATT&CK tactics that have zero failures.

$$\text{CoverageScore} = (\text{tactics with zero failures}) / (\text{total tested tactics}) \times 100$$

A tactic counts as "covered" only when it has been tested and every check within it passed. This metric rewards complete control coverage, not just partial success.

8.5 Trend

Compares PreventionScore of the current run against the most recent prior run for the same agent.

Delta (Current – Previous)	Trend
≥ +5 points	Improving
≤ –5 points	Degrading
Between –5 and +5	Stable
No prior run	Baseline

8.6 Risk Classification

The final **Risk Score** is $\text{round}(\text{ExposureScore} \times \text{KillChainAmplifier})$, clamped to 0–100.

Risk Score	Classification
0–20	Protected
21–40	Low Risk
41–60	Medium Risk
61–80	High Risk
81–100	Critical

8.7 Critical Failures

Any technique with severity **Critical** or **High** that fails is listed in the **CriticalFailures** array, regardless of overall score. This ensures high-priority findings are never buried in aggregate metrics.

9. User Roles & Access Control

The platform enforces role-based access control (RBAC) with three roles. Roles are assigned per user and cannot be combined.

9.1 Role Matrix

Capability	Viewer	Analyst	Admin
View agent list	✓	✓	✓
View scenario list	✓	✓	✓
View scenario run history	✓	✓	✓
View reports & scores	✓	✓	✓
Download agent binaries	✓	✓	✓
Run scenarios		✓	✓
Trigger full scan		✓	✓
View connection config (secret)			✓
Manage users			✓
Reset passwords			✓
Platform settings			✓

9.2 Default Admin Account

On first deployment, the platform creates a default admin account:

- **Username:** **admin**
- **Password:** Configurable via **BAS_ADMIN_PASSWORD** environment variable (default: **ChangeMe!2024**)
- The admin is forced to change the password on first login.

9.3 Password Policy

- Minimum 8 characters
 - Hashed with bcrypt (cost factor 12)
 - Admin can force a password reset on any user account
 - **must_change_pw** flag forces password change before accessing any other view
-

10. Installation & Deployment

10.1 Delivery Package Contents

The delivery ZIP (`bas-install-<version>.zip`) contains:

```
bas-install-<version>/
├── images/
│   ├── bas-orchestrator-<version>.tar    # Orchestrator Docker image
│   ├── postgres-16-alpine.tar           # PostgreSQL Docker image
│   └── caldera-latest.tar                # Caldera (if included)
├── setup.sh                             # Interactive installer
├── docker-compose.yml                   # Compose configuration
├── docker-compose.prod.yml              # Production overrides
├── .env.example                         # Environment variable template
├── scenarios/                           # Scenario YAML library
├── wwwroot/                             # Web dashboard SPA
├── systemd/bas-compose.service          # Systemd service for auto-start
├── VERSION                             # Build version info
└── <customer-id>.lic                    # Customer-specific license
```

10.2 Server Installation

Perform on: `mhsscoe123@ubuntu` (orchestrator server)

Step 1: Transfer the ZIP

```
# From Windows build machine [Windows]
scp dist\bas-install-1.6.0.zip mhsscoe123@192.168.85.128:~/
```

Step 2: Extract and load images

```
# [mhsscoe123@ubuntu]
unzip bas-install-1.6.0.zip
cd bas-install-1.6.0

# Load Docker images (no internet required)
docker load < images/bas-orchestrator-1.6.0.tar
docker load < images/postgres-16-alpine.tar
docker load < images/caldera-latest.tar    # if included
```

Step 3: Configure environment

```
# [mhsscoe123@ubuntu]
cp .env.example .env
nano .env
```

Minimum required variables in `.env`:

```
JWT_SECRET=<random-64-char-string>
AGENT_SECRET=<random-32-char-string>
BAS_ADMIN_PASSWORD=<strong-initial-password>
DATABASE_URL=postgres://bas:bas@postgres:5432/bas?sslmode=disable
```

Step 4: Start the platform

```
# [mhsscoe123@ubuntu]
docker compose up -d
```

Step 5: Install as systemd service (optional, recommended for production)

```
# [mhsscoe123@ubuntu]
sudo cp systemd/bas-compose.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable bas-compose
sudo systemctl start bas-compose
```

Step 6: Verify

```
# [mhsscoe123@ubuntu]
curl http://localhost:9000/health
# Expected: {"status":"ok"}
```

Step 7: Access the dashboard

Open a browser and navigate to <http://<server-ip>:9000>. Log in with the admin credentials configured in Step 3.

10.3 Upgrading

```
# [Windows] Build new version
.\packaging\windows-build.ps1 -Version "1.7.0" -Customer "HDFC Bank" -CustomerID
"hdfc-prod-001" -Days 365

# [mhsscoe123@ubuntu] Load new image and restart
docker load < images/bas-orchestrator-1.7.0.tar
docker compose up -d --no-deps orchestrator
```

The PostgreSQL database is preserved across upgrades. Schema migrations run automatically on startup.

10.4 TLS / HTTPS (Recommended for Production)

The orchestrator listens on plain HTTP by default. For production, place a reverse proxy (nginx, Caddy, or Traefik) in front:

```
server {
    listen 443 ssl;
    server_name bas.internal;

    ssl_certificate      /etc/ssl/bas.crt;
    ssl_certificate_key  /etc/ssl/bas.key;

    location / {
        proxy_pass http://127.0.0.1:9000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
    }
}
```

Important: When TLS is in use, update the `BAS_SERVER_URL` in all agent configs from `http://` to `https://` and ensure WebSocket connections use `wss://`.

11. Agent Deployment

11.1 Windows Agent

Method 1: Download from Dashboard

1. Log in to the dashboard as Admin or Analyst.
2. Navigate to **Agents** → **Download Agent**.
3. Select **Windows (amd64)** and download `bas-agent-windows-amd64.exe`.
4. Transfer to the target Windows machine.

Method 2: PowerShell (from admin prompt on target machine)

```
# [Target Windows]
# Download
Invoke-WebRequest http://<server-ip>:9000/api/agents/download/windows-amd64 -
OutFile bas-agent.exe

# Install as Windows service
.\bas-agent.exe --install --server http://<server-ip>:9000 --env Production

# Verify service running
Get-Service bas-agent
```

11.2 Linux Agent — Debian/Ubuntu (Recommended)

```
# [Target Linux]
# Download .deb package
wget http://<server-ip>:9000/api/agents/download/linux-amd64-deb -O bas-agent.deb

# Install – interactive prompts will ask for server URL and agent secret
sudo dpkg -i bas-agent.deb

# Follow prompts:
#   Orchestrator URL   (e.g. http://192.168.1.10:9000) : http://<server-ip>:9000
#   Agent Secret       (from dashboard → Agents page) : <your-agent-secret>

# Verify
sudo systemctl status bas-agent
```

11.3 Linux Agent — RHEL/CentOS

```
# [Target Linux]
# Download .rpm package
wget http://<server-ip>:9000/api/agents/download/linux-amd64-rpm -O bas-agent.rpm

# Install
sudo rpm -i bas-agent.rpm

# Configure manually (RPM does not run interactive postinst)
sudo nano /etc/bas-agent/config
# Set:
#   BAS_SERVER_URL=http://<server-ip>:9000
#   BAS_AGENT_SECRET=<your-agent-secret>
#   BAS_ENV_LABEL=Production

sudo systemctl daemon-reload
sudo systemctl enable bas-agent
sudo systemctl start bas-agent
```

11.4 Linux Agent — Raw Binary

```
# [Target Linux]
wget http://<server-ip>:9000/api/agents/download/linux-amd64 -O bas-agent
chmod +x bas-agent
sudo BAS_SERVER_URL=http://<server-ip>:9000 BAS_AGENT_SECRET=<secret> ./bas-agent
--install
```

11.5 macOS Agent

```
# [Target macOS]
curl -o bas-agent http://<server-ip>:9000/api/agents/download/darwin-amd64
# For Apple Silicon:
# curl -o bas-agent http://<server-ip>:9000/api/agents/download/darwin-arm64
chmod +x bas-agent
sudo BAS_SERVER_URL=http://<server-ip>:9000 BAS_AGENT_SECRET=<secret> ./bas-agent
--install
```

11.6 Agent Configuration File

The Linux agent stores its configuration in `/etc/bas-agent/config` (mode 600):

```
BAS_SERVER_URL=http://192.168.85.128:9000
BAS_ENV_LABEL=Production
BAS_AGENT_SECRET=<shared-secret>
```

After editing, restart the service:

```
sudo systemctl restart bas-agent
```

11.7 Finding the Agent Secret

1. Log in to the dashboard as **Admin**.
2. Navigate to **Agents** page.
3. Scroll to the **Connection Configuration** card.
4. The Server URL is shown automatically. Click **Show** to reveal the Agent Secret, or **Copy** to copy it to clipboard.

11.8 Viewing Agent Logs

```
# [Target Linux]
journalctl -u bas-agent -f

# [Target Windows - PowerShell]
Get-EventLog -LogName Application -Source bas-agent -Newest 50
```

11.9 Uninstalling the Agent

```
# Linux (.deb)
sudo dpkg -r bas-agent

# Linux (.rpm)
sudo rpm -e bas-agent
```

```
# Windows
.\bas-agent.exe --uninstall
```

12. Dashboard & Operations Guide

12.1 Logging In

Navigate to <http://<server-ip>:9000> in a supported browser. The login screen uses a split-panel layout with platform branding on the left and the authentication form on the right.

On first login with the default admin account, you will be redirected to a mandatory password change screen.

12.2 Dashboard View

The main dashboard shows:

- **Agent Status Summary:** Count of Online / Offline / Scanning agents
- **Recent Run History:** Last 10 scenario runs with status and score
- **Critical Failures:** Aggregated high-severity failures across all recent runs
- **Trend Indicators:** Whether security posture is Improving, Degrading, or Stable

12.3 Running a Scenario

1. Click **Scenarios** in the left sidebar.
2. Browse or filter scenarios by MITRE tactic tag.
3. Click on a scenario card to expand details (framework, steps, techniques covered).
4. Click **Run** on the target agent card, or use the **Run on Agent** dropdown.
5. The scenario is dispatched immediately. Switch to **Live Runs** to watch real-time output.

Note: Running scenarios requires the Analyst or Admin role.

12.4 Interpreting Results

Each step in a run is displayed with:

- **Status badge:** PASS (green), FAIL (red), SKIP (grey), BLOCKED (blue)
- **Technique:** MITRE ATT&CK ID and name
- **Tactic:** Kill-chain phase
- **Severity:** Critical / High / Medium / Low
- **Details:** What the check found
- **Remediation:** Recommended fix for failures
- **Duration:** Execution time in milliseconds

At the top of each completed run:

Metric	Description
Prevention Score	Weighted pass rate (0–100)

Metric	Description
Exposure Score	Weighted fail rate × amplifier (0–100)
Coverage Score	% of tested tactics with zero failures
Risk Classification	Protected / Low / Medium / High / Critical
Trend	Direction vs. previous run

12.5 Tactic Breakdown

The tactic breakdown panel shows per-tactic pass/fail counts, allowing analysts to immediately identify which kill-chain phases have gaps. Tactics are sorted by exposure weight (highest risk first).

12.6 Managing Agents

The **Agents** view shows all registered endpoints with:

- Hostname and IP address
- Operating system and version
- Logged-in username
- Environment label (Production, Staging, etc.)
- Last heartbeat time
- Status (Online / Offline / Scanning)
- Binary trust status (Trusted / Untrusted)

Agents that have not sent a heartbeat for over 90 seconds are automatically marked **Offline**.

12.7 User Management (Admin Only)

Navigate to **Administration** → **Users**.

Creating a user:

1. Click **New User**.
2. Enter username, initial password, and role (Viewer / Analyst / Admin).
3. The user will be prompted to change their password on first login.

Resetting a password:

1. Click the user row.
2. Click **Reset Password** and provide a new temporary password.
3. The user is flagged `must_change_pw = true` and must set a new password on next login.

Deactivating a user: Toggle the **Active** switch on the user row. Deactivated users cannot log in but their history is preserved.

13. API Reference

All API endpoints are served from the orchestrator at `http://<server>:9000`.

13.1 Authentication

Most endpoints require a valid JWT, either as an HttpOnly cookie (**bas_token**) set by the login endpoint, or as a **Bearer <token>** header.

POST /api/auth/login

```
// Request
{ "username": "admin", "password": "your-password" }

// Response 200
{ "token": "<jwt>", "role": "admin", "username": "admin" }
```

POST /api/auth/logout — Clears the session cookie. No request body.

13.2 Agent Endpoints

Method	Path	Auth	Description
GET	/api/agents	JWT (Viewer+)	List all agents
GET	/api/agents/download/{platform}	JWT (Viewer+)	Download agent binary

Platform values: linux-amd64, linux-arm64, linux-amd64-deb, linux-arm64-deb, linux-amd64-rpm, windows-amd64, darwin-amd64, darwin-arm64

13.3 Scenario Endpoints

Method	Path	Auth	Description
GET	/api/scenarios	JWT (Viewer+)	List all scenarios
GET	/api/scenarios/{id}	JWT (Viewer+)	Get scenario detail
POST	/api/scenarios/{id}/run	JWT (Analyst+)	Dispatch to agent
POST	/api/scan/{agentId}	JWT (Analyst+)	Run full-scan scenario
GET	/api/scenarios/runs	JWT (Viewer+)	Query run history

Run request body:

```
{ "agentId": "<agent-id>" }
```

13.4 Reporting Endpoints

Method	Path	Auth	Description
GET	/api/report/{agentId}	JWT (Viewer+)	Get latest report for agent

13.5 Admin Endpoints

Method	Path	Auth	Description
GET	/api/config/connection	JWT (Admin)	Get agent secret
GET	/api/users	JWT (Admin)	List users
POST	/api/users	JWT (Admin)	Create user
PUT	/api/users/{id}	JWT (Admin)	Update user
DELETE	/api/users/{id}	JWT (Admin)	Delete user
POST	/api/users/{id}/reset-password	JWT (Admin)	Force password reset

13.6 Agent-Facing Endpoints (Internal)

These are called by the agent binary and are authenticated with the AGENT_SECRET header.

Method	Path	Auth	Description
POST	/api/heartbeat	X-Agent-Token	Register/update agent status
POST	/api/report	X-Result-MAC	Submit full simulation report
POST	/api/scenarios/result	X-Result-MAC	Submit scenario step results
GET	/ws/agent	WS + agentSecret	WebSocket for command delivery

13.7 Health Check

Method	Path	Auth	Description
GET	/health	None	Liveness check

```
// Response 200
{ "status": "ok" }
```

14. Configuration Reference

14.1 Orchestrator Environment Variables

Variable	Required	Default	Description
DATABASE_URL	Yes	—	PostgreSQL connection string
JWT_SECRET	Yes	—	JWT signing key (min 32 chars)
AGENT_SECRET	No	—	Shared token for agent MAC verification
HTTP_PORT	No	9000	Listening port

Variable	Required	Default	Description
SCENARIOS_DIR	No	scenarios/	Path to scenario YAML files
ART_DIR	No	/art-atomics	Path to ART atomics YAML (bundled in image)
CALDERA_URL	No	—	Caldera REST API URL
CALDERA_API_KEY	No	—	Caldera API authentication key
BAS_LICENSE_PATH	No	—	Path to customer license file
BAS_ADMIN_PASSWORD	No	ChangeMe!2024	Initial admin password

14.2 Agent Environment Variables

Variable	Required	Default	Description
BAS_SERVER_URL	Yes	—	Orchestrator URL (e.g., <code>http://192.168.1.10:9000</code>)
BAS_AGENT_SECRET	No	—	Must match server's <code>AGENT_SECRET</code>
BAS_ENV_LABEL	No	Production	Label shown in dashboard
BAS_AGENT_ID	No	auto-generated	Override agent identity

14.3 Agent Command-Line Flags

Flag	Description
<code>--install</code>	Register as system service
<code>--uninstall</code>	Remove system service
<code>--server <url></code>	Override <code>BAS_SERVER_URL</code>
<code>--env <label></code>	Override <code>BAS_ENV_LABEL</code>

14.4 Scenario YAML Fields

Field	Type	Description
<code>id</code>	string (required)	Unique scenario identifier
<code>name</code>	string (required)	Display name
<code>description</code>	string	Brief description
<code>author</code>	string	Author name
<code>tags</code>	[]string	Filter tags
<code>mitre_phases</code>	[]string	MITRE tactic phases
<code>local_check</code>	bool	Use agent built-in checks

Field	Type	Description
<code>caldera_adversary_id</code>	string	Caldera adversary UUID
<code>caldera_abilities</code>	[]string	Caldera ability IDs
<code>caldera_all_windows</code>	bool	All Caldera Windows abilities
<code>art_techniques</code>	[]string	ART technique IDs
<code>art_all_windows</code>	bool	All ART Windows techniques
<code>steps</code>	[]Step	Static execution steps

Step fields:

Field	Type	Description
<code>name</code>	string	Step display name
<code>technique_id</code>	string	MITRE technique ID (e.g., T1059.001)
<code>framework</code>	string	<code>art</code> , <code>caldera</code> , or <code>custom</code>
<code>executor</code>	string	<code>powershell</code> , <code>cmd</code> , <code>bash</code> , <code>wmi</code> , etc.
<code>command</code>	string	Command to execute
<code>timeout_sec</code>	int	Execution timeout (default: 120)
<code>payloads</code>	[]Payload	Files to stage (name + base64 content)
<code>cleanup</code>	string	Command to run after execution
<code>ability_id</code>	string	Caldera ability UUID (Caldera framework)
<code>test_index</code>	int	ART test variant index (ART framework)

15. Security & Compliance Posture

15.1 Platform Security Controls

Control	Implementation
Authentication	JWT HS256, 24-hour TTL, HttpOnly cookies
CSRF Protection	SameSite=Strict cookie policy
Password Storage	Bcrypt (cost factor 12)
Data in Transit	Recommend TLS via reverse proxy
Result Integrity	HMAC-SHA256 over all agent-submitted results
Binary Integrity	SHA256 manifest verification on agent heartbeat
Code Obfuscation	Garble (<code>-literals</code> <code>-tiny</code>) applied to orchestrator binary

Control	Implementation
Minimal Attack Surface	Distroless base image (no shell, no package manager)
Least Privilege	Orchestrator runs as nonroot user inside container
RBAC	Three-role model (Admin / Analyst / Viewer)
Input Validation	Scenario IDs and ability IDs validated with regex

15.2 Data Classification

Data Type	Location	Sensitivity
Agent results (stdout/stderr)	PostgreSQL scenario_runs.results	Internal
Agent credentials (secret)	Orchestrator env var	Confidential
User passwords	PostgreSQL users.password_hash	Bcrypt hash only
JWT signing key	Orchestrator env var	Confidential
License file	Filesystem	Confidential
Raw command output	Truncated to 3000 chars in DB	Internal

15.3 Network Security Recommendations

1. **Isolate the orchestrator server** on a dedicated management VLAN.
2. **Restrict port 9000** access to analyst workstations and target endpoints only.
3. **Enable TLS** with a valid internal CA certificate (see Section 10.4).
4. **Set a strong AGENT_SECRET** — minimum 32 random characters. This secret is used for MAC verification of all results.
5. **Set a strong JWT_SECRET** — minimum 64 random characters.
6. **Rotate secrets** at each engagement or on a defined schedule.

15.4 Regulatory Alignment

The platform's scenario library includes dedicated coverage for:

- **RBI Cyber Security Framework** — CSCRF/MII drill scenario
- **SEBI Cybersecurity Circular** — Compliance drill checks
- **CIS Benchmarks** — CIS Ubuntu L1 benchmark scenario
- **MITRE ATT&CK** — All scenarios mapped to technique and tactic

16. Licensing

16.1 License Model

Audspect BAS uses node-locked, time-limited licenses. Each license is generated for a specific Customer ID and includes an expiry date. The platform validates the license on startup.

16.2 License File

The license file (`<customer-id>.lic`) is included in the delivery ZIP and must be placed at the path specified in the `BAS_LICENSE_PATH` environment variable.

16.3 License Expiry

When a license expires, the platform will continue serving existing data but will block new scenario executions. Contact support@audspect.com for license renewal.

16.4 Generating a License (Internal — Audspect Use Only)

```
# [Windows - build machine]
.\packaging\windows-build.ps1 `
  -Version "1.6.0" `
  -Customer "HDFC Bank" `
  -CustomerID "hdfc-prod-001" `
  -Days 365
```

17. Troubleshooting

17.1 Agent Not Appearing in Dashboard

Symptom	Likely Cause	Fix
Agent not showing after install	Wrong <code>BAS_SERVER_URL</code>	Verify URL in <code>/etc/bas-agent/config</code> ; must be <code>http://<server-ip>:9000</code> (not localhost)
Agent shows Offline immediately	Port 9000 blocked	Check firewall on both server and client: <code>telnet <server-ip> 9000</code>
401 on heartbeat in agent logs	Wrong or missing <code>BAS_AGENT_SECRET</code>	Copy secret from dashboard Admin → Connection Config and update config

Diagnose on Linux agent:

```
journalctl -u bas-agent -f
```

Diagnose on Windows agent:

```
Get-EventLog -LogName Application -Source bas-agent -Newest 20
```

17.2 Scenario Run Stuck at "Running"

Symptom	Likely Cause	Fix
Run stays in "Running" state	Agent went offline mid-run	Wait 90s — staleness monitor marks it Partial automatically
Run dispatched but no output	WebSocket connection dropped	Check agent logs; reconnect is automatic
All steps show duration 0ms	Timeout too short	Increase <code>timeout_sec</code> in scenario YAML

17.3 Exposure Score and Coverage Score Show 0

This occurs when simulation results have empty tactic assignments. This was caused by a task ID mismatch between the agent and orchestrator (fixed in commit [5284f67](#)). Ensure you are running agent version built after this fix and rebuild the Docker image.

Verify by inspecting a run's results: if `technique.tactic` is empty for most steps, the agent binary predates the fix.

17.4 Docker Build Consuming Excessive Disk

Each Docker build with Garble obfuscation produces non-deterministic binaries (different hash each build = no layer cache reuse). Additionally, image `.tar` files in the `dist\` directory accumulate.

Reclaim disk space [Windows]:

```
# Remove old delivery packages
Remove-Item -Recurse -Force dist\

# Remove dangling Docker image layers
docker image prune -f

# Remove all unused Docker objects (more aggressive)
docker system prune -f
```

17.5 Dashboard WebSocket Not Connecting

Symptoms: Agent list never updates, "Connecting..." status in dashboard.

1. Verify the browser is not behind a proxy that strips `Upgrade` headers.
2. Check the browser console for WebSocket errors.
3. If using a reverse proxy (nginx), ensure the `Upgrade` and `Connection` headers are forwarded (see Section 10.4).

17.6 .deb Package Installed but Agent Not Starting

```
# Check service status
sudo systemctl status bas-agent
```

```
# Check config
sudo cat /etc/bas-agent/config

# Fix: edit with correct values
sudo nano /etc/bas-agent/config
# Set BAS_SERVER_URL and BAS_AGENT_SECRET, then:
sudo systemctl restart bas-agent
```

17.7 Resetting the Admin Password

If the admin password is lost, restart the orchestrator with a new `BAS_ADMIN_PASSWORD` environment variable. The startup routine will reset the admin account password and flag `must_change_pw = true`.

18. Glossary

Term	Definition
ART	Atomic Red Team — open-source library of adversary simulation test cases from Red Canary
Caldera	Open-source adversary emulation platform from MITRE
BAS	Breach and Attack Simulation — methodology for continuously testing security controls against real attack behaviors
BFSI	Banking, Financial Services, and Insurance
Coverage Score	Percentage of tested MITRE ATT&CK tactics where every check passed
Critical Failure	A simulation step with Critical or High severity that failed
DEB	Debian package format used by Ubuntu/Debian Linux
EDR	Endpoint Detection & Response — security software monitoring endpoints for threats
Exposure Score	Tactic-weighted measure of how exposed an environment is (higher = worse)
Framework	Execution method for a scenario step: <code>art</code> , <code>caldera</code> , or <code>custom</code>
Garble	Go code obfuscation tool that renames symbols and obfuscates string literals
Heartbeat	Periodic status message from agent to orchestrator (every 30 seconds)
HMAC	Hash-based Message Authentication Code — used to sign agent result submissions
JWT	JSON Web Token — used for browser session authentication
Kill Chain	Ordered sequence of ATT&CK tactics representing an attacker's progression
Kill Chain Amplifier	Score multiplier applied when multiple consecutive kill-chain phases fail

Term	Definition
Local Check	Scenario that uses the agent's built-in posture checks rather than remote framework commands
LOLBIN	Living-Off-the-Land Binary — legitimate OS binary abused by attackers
MITRE ATT&CK	Knowledge base of adversary tactics, techniques, and procedures
Orchestrator	The central BAS server: receives results, manages agents, scores runs, serves dashboard
Prevention Score	Severity-weighted measure of how well controls are stopping attacks (higher = better)
RPM	Red Hat Package Manager format used by RHEL/CentOS Linux
Scenario	A named collection of attack simulation steps, defined in YAML
ScenarioRun	A single execution of a scenario on a specific agent
SimulationResult	The normalized output of a single scenario step: technique, result, severity, details, remediation
Tactic	MITRE ATT&CK kill-chain phase (e.g., credential-access, lateral-movement)
Technique	Specific MITRE ATT&CK method within a tactic (e.g., T1003.001 — LSASS Memory)
Trend	Direction of PreventionScore change vs. previous run (Improving/Degrading/Stable/Baseline)
TTP	Tactics, Techniques, and Procedures — the vocabulary of adversary behavior

© Audspect — All rights reserved.

This document is confidential and intended solely for the named customer. Unauthorized distribution is prohibited.

Document History

Version	Date	Description
1.0	2026-06-01	Initial release — platform v1.6.0